

3.1 Task 1: Generate an Encryption Key the Wrong Way

1. By compiling the provided code into a file named time.c, I was able to execute the code and observe the results:

```
[01/21/26] seed@VM:~/A1$ ./time
1769022156
fe01133fa0b7f17f3106ad14e0072adf
[01/21/26] seed@VM:~/A1$ ./time
1769022157
3a09a2a975ebdc7e7b5de9ec1ef7c149
```

Running this program back to back with only a difference of 1 between the time values provided, the resulting keys are vastly unlike. Since in these two instances the seed used to create `srand(t)` is different, the values created are expected to be vastly different

On the other hand if I were to run `./time` twice in the same second, I should expect the outputs to be identical, since the seed for `srand()` would be the same and the first 16 values of `rand()` would then be identical:

```
[01/21/26] seed@VM:~/A1$ ./time
1769022429
906fe94d5f7c26854807b57219cd5cea
[01/21/26] seed@VM:~/A1$ ./time
1769022429
906fe94d5f7c26854807b57219cd5cea
```

2. By commenting out the `srand()` definition, the same key is generated no matter the time:

```
[01/21/26] seed@VM:~/A1$ ./time
1769023116
67c6697351ff4aec29cdbaabf2fbe346
[01/21/26] seed@VM:~/A1$ ./time
1769023116
67c6697351ff4aec29cdbaabf2fbe346
[01/21/26] seed@VM:~/A1$ ./time
1769023120
67c6697351ff4aec29cdbaabf2fbe346
```

The key generated is the same each time since every call of this function will use the same rand seed of 1, which is the default. If I were to uncomment srand() and set the seed to 1 instead of the current time, I would get the same results:

```
srand (1);
```

```
[01/21/26] seed@VM:~/A1$ ./time  
1769023406  
67c6697351ff4aec29cdbaabf2fbe346
```

3. Based on my observations, the use of srand() with a seed based on the current second is used to generate a unique key. srand() is used so that the rand() calls ideally generate different values each time this function is called, and the time() call is used to change the behaviour of srand() every second.

This is not really random though, since the amount of seconds since Epoch isn't a random value, and if someone knew the day or year that you generated a key they could pretty easily brute force guess a range of values (There have been ~32 million seconds in the past year. 32 million guesses would not take very long to crack.)

3.2 Task 2: Guess the Key

1.

a. encrypt() function:

```
int encrypt(unsigned char *plaintext, int plaintext_len,
            unsigned char *key, unsigned char *iv, unsigned char *ciphertext) {
    int len;
    int ciphertext_len;
    EVP_CIPHER_CTX *ctx;

    if(!(ctx = EVP_CIPHER_CTX_new())) {
        return -1;
    }

    if (1 != EVP_EncryptInit_ex(ctx, EVP_aes_128_cbc(), NULL, key, iv)) {
        EVP_CIPHER_CTX_free(ctx);
        return -1;
    }

    if (1 != EVP_EncryptUpdate(ctx, ciphertext, &len, plaintext, plaintext_len)) {
        EVP_CIPHER_CTX_free(ctx);
        return -1;
    }

    ciphertext_len = len;

    if (1 != EVP_EncryptFinal_ex(ctx, ciphertext + len, &len)) {
        EVP_CIPHER_CTX_free(ctx);
        return -1;
    }

    ciphertext_len += len;
    EVP_CIPHER_CTX_free(ctx);
    return ciphertext_len;
}
```

b. hex_to_bytes() function:

```
void hex_to_bytes(const char *hex, unsigned char *out) {
    size_t len = strlen(hex);
    for (size_t i=0; i<len/2; i++) {
        sscanf(hex + 2*i, "%2hhx", &out[i]);
    }
}
```

c. generate_key() function:

```
void generate_key(unsigned char *key, int seed) {
    srand(seed);
    for (int i=0; i<16; i++) {
        key[i] = rand()%256;
    }
}
```

d. main() function:

```
int main(void) {
    unsigned char *plaintext = "255044462d312e350a25bfff7a2fe0a33";
    unsigned char *iv = "6f6d6567616c756c3132333435363738";
    unsigned char *desired_ciphertext = "b644d7961944854b655f40caadb30eda";

    unsigned char plaintext_bytes[16];
    unsigned char desired_bytes[16];
    unsigned char iv_bytes[16];
    unsigned char key_bytes[16];

    hex_to_bytes(plaintext, plaintext_bytes);
    hex_to_bytes(iv, iv_bytes);
    hex_to_bytes(desired_ciphertext, desired_bytes);

    unsigned char new_ciphertext[128];

    int start_time = 1767628800;
    int end_time = 1767762000;
    unsigned char key[16];

    for (int i=start_time; i<end_time; i++) {
        generate_key(key, i);
        int encrypted_len = encrypt(plaintext_bytes, sizeof(plaintext_bytes), key, iv_bytes, new_ciphertext);

        if (memcmp(new_ciphertext, desired_bytes, 16)==0) {
            printf("Found seed! Seed = %d\n", i);
        }
    }
}
```

2. The strategy that I took to crack the key was a brute force algorithm that attempted all seconds as keys from ~24 hours before the file was modified, to ~24 hours after. Since the first 8 blocks of plaintext, iv, and first 8 blocks of ciphertext, I can brute force the key value by encrypting the plaintext with the iv and likely key values until I create a ciphertext that matches the one Bob found. This algorithm tested a key for all ~130,000 seconds surrounding Alice's key's second, which ended up being 1767757009.

To begin, I prepared the plaintext and iv for the AES encryption function by converting them into bytes with the 'hex_to_bytes()' function. This function would also be needed to convert Alice's key value, and was used in the main brute force loop.

Next, I established the start and end time values (in seconds), and created a for loop that looped through each second and attempted encryption with the new key value, the plaintext, and the IV. During each loop, the seconds value was converted into an 8-byte value using Alice's algorithm (algorithm provided in task 1) by the generate_key() function, which is just a mimic of her algorithm. From here, the encryption is performed, and the resulting ciphertext is compared to the one Bob grabbed to see if the 16 bytes he found exist in the newly created ciphertext.

When a match was found, the time that used the key was printed, which was 1767757009

```
[01/21/26] seed@VM:~/A1$ ./bobcrack
Found seed! Seed = 1767757009
```

Convert epoch to human-readable date and vice versa

 [\[batch convert\]](#)

Supports Unix timestamps in seconds, milliseconds, microseconds and nanoseconds.

Assuming that this timestamp is in **seconds**:

GMT : Wednesday, January 7, 2026 3:36:49 AM

Your time zone : Tuesday, January 6, 2026 10:36:49 PM GMT-05:00

3.3 Task 3: Generating Cryptographic Key

1. To modify the snippet provided to 256 bit and compile it, I can modify the LEN value to 32, and add a main function to run the script (and output the key):

```
#include <stdlib.h>
#include <stdio.h>
#define LEN 32 // 128 bits

int main(void){
    unsigned char *key = (unsigned char *) malloc(sizeof(unsigned char)*LEN);
    FILE* random = fopen("/dev/urandom", "r");
    fread(key, sizeof(unsigned char)*LEN, 1, random);

    for (int i = 0; i < LEN; i++) {
        printf("%02x", key[i]);
    }
    printf("\n");

    fclose(random);
}
```

After getting the program running and modifying the LEN value from 16 to 32, you can see the length of the key double in size as well

```
[01/21/26] seed@VM:~/A1$ gcc -o generate_crypto generate_crypto.c
[01/21/26] seed@VM:~/A1$ ./generate_crypto
068616e25f36bcd8331814177a4a4b6
[01/21/26] seed@VM:~/A1$ ./generate_crypto
fc20425a02300e72b6fc79f42265d9bc
[01/21/26] seed@VM:~/A1$ ./generate_crypto
bb57f0d7dfa63657ef7aa0lead14cdef
[01/21/26] seed@VM:~/A1$ ./generate_crypto
ad72e74a7e56ae46dd74c35e4261333a
[01/21/26] seed@VM:~/A1$ vim generate_crypto.c
[01/21/26] seed@VM:~/A1$ vim generate_crypto.c
[01/21/26] seed@VM:~/A1$ gcc -o generate_crypto generate_crypto.c
[01/21/26] seed@VM:~/A1$ ./generate_crypto
34b2959f9c2df67b0cdd2549f9dd7848d4b067c8161e446104a1228b47249f32
[01/21/26] seed@VM:~/A1$ ./generate_crypto
eff05591d874c4e4d2323583864b63b0bed3ffbc0127b503749bb1c6b290afa2
```

3.4 Task 4: Examining the Kernel's Entropy Sources

```
seed@VM: ~/A1
[01/21/26]seed@VM:~/A1$ grep 'add.*_randomness' /proc/kallsyms
0000000000000000 T add_device_randomness
0000000000000000 t perf_trace_add_device_randomness
0000000000000000 t perf_trace_add_input_randomness
0000000000000000 t perf_trace_add_disk_randomness
0000000000000000 t trace_raw_output_add_device_randomness
0000000000000000 t trace_raw_output_add_input_randomness
0000000000000000 t trace_raw_output_add_disk_randomness
0000000000000000 t _bpf_trace_add_device_randomness
0000000000000000 t _bpf_trace_add_disk_randomness
0000000000000000 t _bpf_trace_add_input_randomness
0000000000000000 t trace_event_raw_event_add_input_randomness
0000000000000000 t trace_event_raw_event_add_device_randomness
0000000000000000 t trace_event_raw_event_add_disk_randomness
0000000000000000 t add_timer_randomness
0000000000000000 T add_input_randomness
0000000000000000 T add_disk_randomness
0000000000000000 T add_hwgenerator_randomness
0000000000000000 T add_bootloader_randomness
0000000000000000 T add_interrupt_randomness
0000000000000000 t add_early_randomness
0000000000000000 r _tpstrtab_add_disk_randomness
0000000000000000 r _tpstrtab_add_input_randomness
0000000000000000 r _tpstrtab_add_device_randomness
0000000000000000 r _ksymtab_add_device_randomness
0000000000000000 r _ksymtab_add_bootloader_randomness
0000000000000000 r _ksymtab_add_disk_randomness
0000000000000000 r _ksymtab_add_hwgenerator_randomness
0000000000000000 r _ksymtab_add_input_randomness
0000000000000000 r _ksymtab_add_interrupt_randomness
0000000000000000 r _kstrtab_add_bootloader_randomness
0000000000000000 r _kstrtab_add_hwgenerator_randomness
0000000000000000 r _kstrtab_add_disk_randomness
0000000000000000 r _kstrtab_add_interrupt_randomness
0000000000000000 r _kstrtab_add_input_randomness
0000000000000000 r _kstrtab_add_device_randomness
0000000000000000 d print_fmt_add_disk_randomness
0000000000000000 d print_fmt_add_input_randomness
0000000000000000 d print_fmt_add_device_randomness
0000000000000000 d trace_event_type_funcs_add_disk_randomness
0000000000000000 d trace_event_type_funcs_add_input_randomness
0000000000000000 d trace_event_type_funcs_add_device_randomness
0000000000000000 d event_add_disk_randomness
0000000000000000 d event_add_input_randomness
0000000000000000 d event_add_device_randomness
0000000000000000 d event_class_add_disk_randomness
0000000000000000 d event_class_add_input_randomness
0000000000000000 d event_class_add_device_randomness
0000000000000000 D _tracepoint_add_disk_randomness
0000000000000000 D _tracepoint_add_input_randomness
0000000000000000 D _tracepoint_add_device_randomness
0000000000000000 d bpf_trace_tp_map_add_disk_randomness
0000000000000000 d bpf_trace_tp_map_add_input_randomness
0000000000000000 d bpf_trace_tp_map_add_device_randomness
0000000000000000 t trace_event_define_fields_add_device_randomness
0000000000000000 t trace_event_define_fields_add_input_randomness
0000000000000000 t trace_event_define_fields_add_disk_randomness
0000000000000000 t _event_add_disk_randomness
0000000000000000 t _event_add_input_randomness
0000000000000000 t _event_add_device_randomness
```

1. From this list of randomness functions, new add_*_randomness calls appear as input, device, disk, hwgenerator, bootloader, and early.
 - a. Input: This would be input devices, like the mouse and keyboard randomness examples provided.

- b. Device: These are functions that measure randomness from the input devices, like network interrupts or timers. These were mentioned as `add_interrupt_randomness` in the assignment, but `add_device_randomness` likely covers the same things.
 - c. Disk: This would be disk read/write timings, and were mentioned briefly in the assignment
 - d. Hwgenerator: These are hardware randomness generators, like RDRAND on some Intel chips.
 - e. Bootloader: Bootloader entropy is collected as the device is starting up, before the OS is loaded
 - f. Early: Similar to bootloader, just entropy collected early on in the computer's bootup
2. By compiling and running the Python input listener, I was able to see all of the `add_interrupt_randomness` calls on my computer during a span of time:

```
[01/21/26]seed@VM:~/A1$ sudo python3 randomness.py
Tracing all main entropy sources... Ctrl-C to stop.
b'    gvfsd-trash-2461 [001] d.h. 22402.115012: 0: add_interrupt_randomness: irq=21'
b'    gvfsd-trash-2461 [001] d.H. 22402.115360: 0: add_interrupt_randomness: irq=21'
b'    gvfsd-trash-2461 [001] dNH. 22402.115731: 0: add_interrupt_randomness: irq=21'
b' systemd-journal-234 [001] dNH. 22402.132904: 0: add_interrupt_randomness: irq=21'
b'    <idle>-0         [000] dNh. 22402.140287: 0: add_interrupt_randomness: irq=18'
b'    <idle>-0         [000] dNh. 22402.141639: 0: add_interrupt_randomness: irq=18'
b'    Xorg-2112        [001] d.h. 22402.152484: 0: add_interrupt_randomness: irq=21'
b'    Xorg-2112        [001] d.h. 22402.154010: 0: add_interrupt_randomness: irq=21'
b'    Xorg-2112        [001] d.h. 22402.154514: 0: add_interrupt_randomness: irq=21'
b'    Xorg-2112        [001] d.h. 22402.164392: 0: add_interrupt_randomness: irq=21'
b'    Xorg-2112        [001] dNH. 22402.165281: 0: add_interrupt_randomness: irq=21'
b'    <idle>-0         [000] dNh. 22402.174644: 0: add_interrupt_randomness: irq=18'
b'    <idle>-0         [001] d.h. 22402.174816: 0: add_interrupt_randomness: irq=21'
b'    <idle>-0         [000] dNh. 22402.176305: 0: add_interrupt_randomness: irq=18'
b'    Xorg-2112        [001] d.h. 22402.176312: 0: add_interrupt_randomness: irq=21'
b'    Xorg-2112        [001] d.h. 22402.177006: 0: add_interrupt_randomness: irq=21'
```

These calls occurred constantly, and also when I pressed buttons on my keyboard or clicked with my mouse.

- By modifying the python file to probe the `add_input_randomness` entropy collector, I could now see the events of my keyboard and mouse clicks:

Typing “hheey” and clicking twice:

```
[01/21/26]seed@VM:~/A1$ sudo python3 randomness.py
Tracing all main entropy sources... Ctrl-C to stop.
b'      <idle>-0      [001] d.h. 24027.629758: 0: add_input_randomness: type=4, code=4, value=35'
b'      <idle>-0      [001] d.h. 24027.630243: 0: add_input_randomness: type=1, code=35, value=1'
hb'     <idle>-0      [001] d.h. 24027.721479: 0: add_input_randomness: type=4, code=4, value=35'
b'      <idle>-0      [001] d.h. 24027.721654: 0: add_input_randomness: type=1, code=35, value=0'
b'      <idle>-0      [001] d.h. 24028.215883: 0: add_input_randomness: type=4, code=4, value=35'
b'      <idle>-0      [001] d.h. 24028.215958: 0: add_input_randomness: type=1, code=35, value=1'
hb'     <idle>-0      [001] d.h. 24028.289587: 0: add_input_randomness: type=4, code=4, value=35'
b'      <idle>-0      [001] d.h. 24028.289658: 0: add_input_randomness: type=1, code=35, value=0'
b'      <idle>-0      [001] d.h. 24028.833178: 0: add_input_randomness: type=4, code=4, value=18'
b'      <idle>-0      [001] d.h. 24028.833331: 0: add_input_randomness: type=1, code=18, value=1'
eb'     <idle>-0      [001] d.h. 24028.949657: 0: add_input_randomness: type=4, code=4, value=18'
b'      <idle>-0      [001] d.h. 24028.949739: 0: add_input_randomness: type=1, code=18, value=0'
b'      <idle>-0      [001] d.h. 24029.388917: 0: add_input_randomness: type=4, code=4, value=18'
b'      <idle>-0      [001] d.h. 24029.388987: 0: add_input_randomness: type=1, code=18, value=1'
eb'     <idle>-0      [001] d.h. 24029.492931: 0: add_input_randomness: type=4, code=4, value=18'
b'      <idle>-0      [001] d.h. 24029.493011: 0: add_input_randomness: type=1, code=18, value=0'
b'      <idle>-0      [001] d.h. 24029.922560: 0: add_input_randomness: type=4, code=4, value=21'
b'      <idle>-0      [001] d.h. 24029.922741: 0: add_input_randomness: type=1, code=21, value=1'
yb'     <idle>-0      [001] d.h. 24030.021963: 0: add_input_randomness: type=4, code=4, value=21'
b'      <idle>-0      [001] d.h. 24030.022031: 0: add_input_randomness: type=1, code=21, value=0'
b'      <idle>-0      [001] d.h. 24030.432889: 0: add_input_randomness: type=4, code=4, value=21'
b'      <idle>-0      [001] d.h. 24030.432955: 0: add_input_randomness: type=1, code=21, value=1'
yb'     <idle>-0      [001] d.h. 24030.527797: 0: add_input_randomness: type=4, code=4, value=21'
b'      <idle>-0      [001] d.h. 24030.527868: 0: add_input_randomness: type=1, code=21, value=0'
b'      <idle>-0      [001] d.h. 24031.207971: 0: add_input_randomness: type=1, code=272, value=1'
b'      <idle>-0      [001] d.h. 24031.270281: 0: add_input_randomness: type=1, code=272, value=0'
b'      <idle>-0      [001] d.h. 24031.769114: 0: add_input_randomness: type=1, code=272, value=1'
b'      <idle>-0      [001] d.h. 24031.851986: 0: add_input_randomness: type=1, code=272, value=0'
```

Here, you can see that each button click/unclick loads 4 lines, and you can see the characters I pressed appearing on the left side of the terminal as well. It seems the code/value has to do with the specific button being pressed, and the type of 4 and 1 show whether it is being pressed down or released.

This is the code I used to detect the input entropy, which is mostly similar to the code provided but uses the `add_input_randomness` and `trace_add_input_randomness` calls:

```
from bcc import BPF

bpf_input_text = """#include <uapi/linux/ptrace.h>

int trace_add_input_randomness(struct pt_regs *ctx, int type, int code, int value)
{
    bpf_trace_printk("add_input_randomness: type=%d, code=%d, value=%d\\n",type,code,value);
    return 0;
}"""

b = BPF(text=bpf_input_text)
b.attach_kprobe(event="add_input_randomness", fn_name="trace_add_input_randomness")
print("Tracing all main entropy sources... Ctrl-C to stop.")
b.trace_print()
```


I found the source code for the `add_input_randomness` function for Linux 5.4 on the Elixir site, which showed me which arguments to use for the function:

v5.4.2	1254	<code>void add_input_randomness(unsigned int type, unsigned int code,</code>
v5.4.1	1255	<code>unsigned int value)</code>
v5.4	1256	<code>{</code>
v5.4-rc8	1257	<code>static unsigned char last_value;</code>
v5.4-rc7	1258	
v5.4-rc6	1259	<code>/* ignore autorepeat and the like */</code>
v5.4-rc5	1260	<code>if (value == last_value)</code>
v5.4-rc4	1261	<code>return;</code>
v5.4-rc3	1262	
v5.4-rc2	1263	<code>last_value = value;</code>
v5.4-rc1	1264	<code>add_timer_randomness(&input_timer_state,</code>
v5.3	1265	<code>(type << 4) ^ code ^ (code >> 4) ^ value);</code>
	1266	<code>trace_add_input_randomness(ENTROPY_BITS(&input_pool));</code>
	1267	<code>}</code>

3.5 Task 5: Breaking a Reused One-Time Pad

1. By inspecting the two ciphertexts, I can identify that the 10th character of both ciphertexts is identical. I know this because the ciphertexts both use the same OTP and are created with a deterministic encryption algorithm, meaning that the only way for the outputs to be the same with identical OTP and algorithms, is for the input to be the same
2. By XOR'ing the two ciphertexts together, I can effectively remove the OTP from the texts, and create the value of $P1 \text{ xor } P2$. Along with the knowledge that the characters in the plaintext are converted to hexadecimal for encryption, and are all lowercase letters, I can do some analysis of the provided values to find patterns. One useful pattern is the value of 'space' on the ASCII table, as well as the possible range that letters can have. By comparing the ASCII values for 'a' and 'z', I know that the largest gap between plaintext letters is 26 (1A in hexadecimal), therefore any values produced in $P1 \text{ xor } P2$ greater than 1A **must** be spaces (or periods and commas, but this isn't mentioned so I assumed not).

Additionally, the binary of the ASCII value of 'space' is 00100000 and the ASCII value for 'a' is 01100000, using this information and my knowledge that there are only 26 lowercase possible letters (increasing linearly from 01100000 to 01111010), I know if there is a 1 in the second position of a binary number, it must have been xor'd with a space (otherwise it would be xor'd away, since all possible letters could result in a 0 in the second position). Using this knowledge, I know there is definitely a space in the 4th, 6th, and 13th positions guaranteed, and possibly a space in the 10th position (though this is unlikely)

3. Knowing that there are spaces in the 4th, 6th, and 13th positions, I can xor space values into these positions of $P1 \text{ xor } P2$ to determine one of the letters. By doing this, I discover that there is an 'r' in one of the 4th positions, an 'n' in one of the 6th positions, and an 's' in one of the 13th positions. Since the plaintexts are sort of intertwined and I only know the distance between their letters, I cannot make any judgements yet as to which sentence these letters and spaces belong to. I know that they are phrases though, and so the spaces will definitely belong to both sentences, rather than all of the spaces in just one.

4. First, I attempted to sort of brute force the algorithm by using common english words and seeing if I could find any recognizable patterns in the outcome:

```
"the the the the " = "guaru&t%hh|<'b}&<+"
" the the the the " = "3il7!:y`<tqys~pc<+"
"  the the the the " = "3=p:dnemy mt6*lny+"
"you you you you " = "jrq+n;hjiyvi*em@s~"
" you you you you " = "3dk'!7~p<yvissws<+"
"  you you you you " = "3=}~tnhji `s&*aii+"
"be be be be be be " = "qx$0dns`<b|<1o8dy+"
" be be be be be be " = " 3@arc+1gy {ysh}&~n"
"of of of of of of " = "|{$=gn~c<o@<<18iz+"
" of of of of of of " = "3rbrn(1jz vzse~&sm"
"to to to to to to " = "gr$&nnej<tv<'e8rs+
" to to to to to to " = "3ikru!1qs mss~w&hd"
"and and and and " = "rs`r` u%}n}<2d|&<+"
" and and and and " = "3|j6!/@a<awxskvb<+"
"  and and and and " = "3=e<enpkx xr7*yhx+"

```

Of these results, the best lead I had was with the first line, as “the “ was a great start to a phrase (lets call this P1), and “guaru” (of P2) used English lowercase letters and seemed like it could be a word. This also works with my knowledge that there is a space in the 4th position, used by P1, so the first word of P2 would likely be 5 letters otherwise P1 would be “the i ...” or “the a ...” which doesn’t seem grammatical.

From here I searched for 5 (and 9) letter words starting with “guar”/”guaru” in the dictionary, and found that “guard” was the most likely candidate.

By xor’ing “guard “ with my existing C1 xor C2 string, I was able to decode some more information about P1:

XOR Calculator

Thanks for using the calculator. [View help page.](#)

I. Input: ASCII (base 256) ▾

guard

II. Input: hexadecimal (base 16) ▾

131d0452014e11051c00191c530a18061c0b

Calculate XOR

III. Output: ASCII (base 256) ▾

the en1%< 9<s*8&<+

Next, I did some more analysis on P1 knowing that the next word started with “en”, and did more analysis with my knowledge of the spaces. With the spaces at positions 4 and 6 now being taken, I knew the next word in P1 started with “en” and was either 5 letters long (if the matching characters in the plaintext is a space), 8 letters long, or 14 letters long (unlikely). I hit the dictionary again, and created a new list of words to test:

"the entry	" = "guard ewe 9<s*8&<+"
"the enter	" = "guard e`n 9<s*8&<+"
"the enemy	" = "guard the 9<s*8&<+"
"the encoding	" = "guard rjxiw{s*8&<+"
"the encrypts	" = "guard rwepmos*8&<+"
"the enhanced	" = "guard ydrc xs*8&<+"

“the enemy “ for P1 fit the best, with P2 using the common word “the” grammatically correct. Furthermore, the sentence “the enemy is “ worked great, giving me “guard the pos*8&<+", and with this I looked for 8 letter words starting with “pos”

“position” was the only word that made sense, and xor’ing P2 (“guard the position”) with the xor’d ciphertexts, I was given the full P1 of “the enemy is close”.